Mikhail Razakov

Compiler Engineer · LLVM · Language Infrastructure

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Compiler engineer with professional LLVM and C# static-analysis experience. At MCST, I worked on LLVM 22 loop/interprocedural analysis, legality, and IR transformations; at ISP RAS, I contribute to SharpChecker, an industrial C#/.NET static-analysis platform. Independent work spans conservative LLVM transformations, Roslyn fixed-point data flow, and UniversalToolchain, a compiler/language-infrastructure project built around typed artifact contracts and deterministic planning; an architecture talk based on the project was accepted to LangDev'26.

Professional compiler / analysis

MCST: LLVM passes, loop/interprocedural analysis, and legality; ISP RAS: SharpChecker C#/.NET static analysis.

Compiler architecture

UniversalToolchain: typed artifacts; dependencies/conflicts, provider selection, pass order, artifact routes, and backends compile into an immutable LanguagePlan before execution.

Verification & backends

Conservative rejection paths, LLVM Verifier and before/after execution; interpreter/CIL parity; a separate x86-64 codegen/register-allocation pipeline.

EXPERIENCE

2026 — present

ISP RAS — Static Analysis Engineer

- Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#/.NET.

July — August 2026

MCST — Compiler Engineering Intern

- Implemented an LLVM LICM pass with loop analysis, side-effect/speculative-safety checks, and hoisting of loop-invariant instructions to preheaders.
- Developed a conservative interprocedural global-IV prototype with APInt affine evolution, transitive call effects, a separate legality phase, and LLVM IR transformation; unsupported CFG/call cases are rejected.

PROJECTS

LLVM Interprocedural Global IV Optimization

A conservative LLVM 22 prototype/MVP with an actual IR transform; 29 positive/negative regression cases cover verifier validity, idempotence, and before/after execution.

UniversalToolchain

LanguageRuntime validates and materializes the exact planned package/component graph without re-planning; plan-determinism/exact-binding checks, ownership guards, and interpreter/CIL parity protect the contract.

DerefAfterNullAnalyzer

Diagnostics for potentially unsafe dereference paths over the real CFG, with successor reprocessing until the data-flow state reaches a fixed point.

x86-64 Codegen & Register Allocation Lab

A reference interpreter and differential execution check generated-code semantics; the allocator assignment contract is verified independently.

SKILLS

Compilers

C++, LLVM 22, LLVM IR, optimization passes, interprocedural analysis, CFG/SSA, dominators, loop analysis, LICM, legality analysis

Static / Program Analysis

C#, .NET, Roslyn ControlFlowGraph, fixed-point data-flow analysis, state propagation and joins

Compiler Infrastructure

typed artifact contracts, IR/lowering pipelines, deterministic planning/pass ordering, backend/runtime composition, executable verification

Codegen & Systems

Rust, SysV x86-64, liveness/interference, register allocation, Linux, CMake

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

Moscow / remote